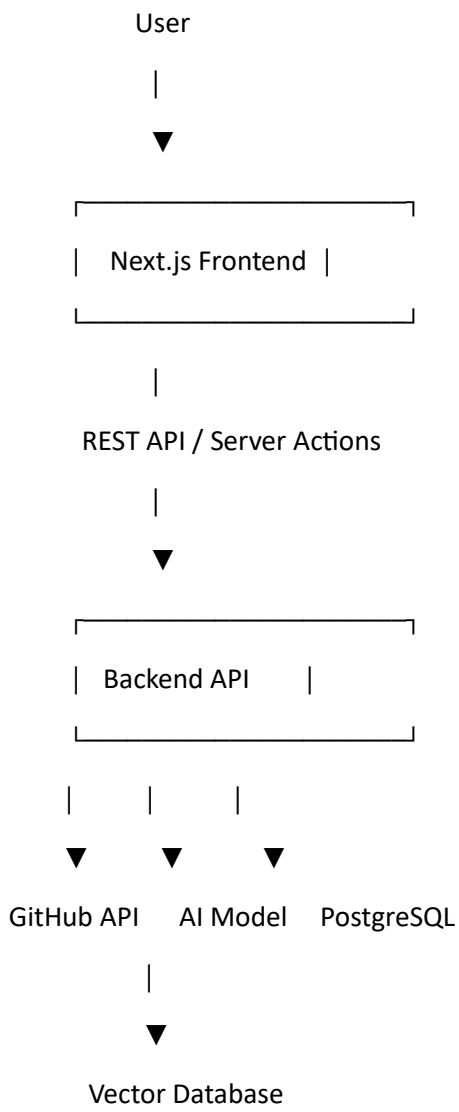
I think we can build this as if you're working in a real software company. Instead of just copying code, you'll understand **why** every technology is used.

This project will teach you **React + Next.js + Node.js + AI + RAG + GitHub APIs + Databases + Authentication + Deployment**.

Project Architecture



Tech Stack

Frontend

- Next.js
- React
- Tailwind CSS
- TypeScript

- React Flow
- Monaco Editor

Backend

- Next.js API Routes (recommended)
- TypeScript

Database

- PostgreSQL
- Prisma ORM

AI

- Gemini API (free tier to start)
- Later:
    - OpenAI
    - Claude

RAG

- LangChain
- Qdrant or Chroma

Authentication

- NextAuth/Auth.js
- GitHub Login

Folder Structure

github-ai/

app/

components/

lib/

services/

prisma/

public/

types/

utils/

Step 1 — Learn Next.js

Before building anything learn

✅ Routing

app/

page.tsx

dashboard/page.tsx

repository/page.tsx

Learn

- App Router
- Layout
- Components
- Server Components
- Client Components

Time

2 Days

Step 2 — Build the UI

Design:

GitHub AI Explainer

Paste Repository URL

_____

Analyze

Don't use AI yet.

Just frontend.

Learn

- Forms
- Tailwind
- Components
- State

Time

2 Days

Step 3 — GitHub API

Now user enters

https://github.com/facebook/react

Extract

facebook

react

Call

GET

/repos/facebook/react

Receive

Stars

Forks

Language

Description

Owner

License

Display on screen.

Now you learned

- REST API
- Fetch
- JSON

Step 4 — Folder Structure

Use GitHub API

GET

/repos/{owner}/{repo}/contents

Now display

src/

components/

hooks/

README.md

package.json

Learn

Recursive programming.

Step 5 — Repository Tree

Build

src

├— components

├— pages

├— hooks

Use recursion.

You'll finally understand recursion in a real project.

Step 6 — Explain Repository

Collect

README

package.json

Folder names

Description

Send to Gemini

Prompt

Explain this project to a beginner.

Mention

Purpose

Framework

Architecture

Folder structure

Main features

Gemini replies

This project is...

Display.

Congratulations.

You built your first AI feature.

Step 7 — Explain Folder

Click

src

Now ask AI

Explain the purpose of this folder.

Output

Contains application source code.

Step 8 — Explain File

Click

Auth.ts

Read file.

Send

Explain this code.

Output

Purpose

Functions

Flow

Dependencies

Now your project becomes useful.

Step 9 — Markdown Viewer

Render

README

Beautifully.

Learn

Markdown.

Step 10 — Code Viewer

Open file

Highlight syntax

Use

Monaco Editor

Learn

Code Editors

Step 11 — Authentication

GitHub Login

Save

User

Projects

History

Learn

OAuth

JWT

Sessions

Step 12 — Database

Tables

Users

Repositories

History

Store

Who analyzed

When

Repository URL

Summary

Learn

Prisma

PostgreSQL

Relationships

Step 13 — Search

Search previous repositories.

Learn

Database querying

Filtering

Pagination

Step 14 — AI Chat

Now user asks

How authentication works?

Search repository.

Give AI only relevant files.

AI answers.

Congratulations.

You built RAG.

Step 15 — Embeddings

Convert

README

Files

Functions

Into vectors.

Store

Qdrant

or

Chroma

Now semantic search becomes fast.

Step 16 — Architecture Diagram

Generate

Frontend

↓

Backend

↓

Database

Later

Mermaid

graph TD

A-->B

B-->C

Step 17 — README Generator

AI writes

Installation

Usage

API

Folder structure

Contributors

Step 18 — Code Review

AI checks

Complexity

Performance

Naming

Security

Step 19 — Deployment

Frontend

Vercel

Backend

Vercel

Database

Neon PostgreSQL

Vector DB

Qdrant Cloud

Step 20 — Final Features

Add

- Dark mode
- Export PDF
- Share analysis
- History

- Multi-language
- Compare repositories
- Team workspaces
- Pull request reviews

What You'll Learn at Each Stage

Stage	Skills
1	Next.js, React, Tailwind, TypeScript
2	Forms, Components, State Management
3	REST APIs, JSON, Async Fetching
4	GitHub API, Recursive File Traversal
5	Tree Data Structures, Recursion
6	Prompt Engineering, LLM Integration
7	Context-Aware AI Explanations
8	Code Parsing and Display
9	Markdown Rendering
10	Syntax Highlighting and Code Editors
11	OAuth, Authentication, Sessions
12	PostgreSQL, Prisma ORM
13	Searching, Pagination, Database Queries
14	RAG, Embeddings, Semantic Search
15	Vector Databases
16	Graphs and Visualization
17	AI Documentation Generation
18	Static Analysis and AI Code Review
19	Cloud Deployment
20	Product Polish and Advanced Features

I recommend building this in six milestones instead of twenty isolated steps:

1. **Foundation:** Next.js, Tailwind, GitHub URL input, repository metadata.
2. **Repository Explorer:** Folder tree, file browser, README rendering, code viewer.
3. **AI Explainer:** Repository, folder, and file explanations using Gemini.
4. **AI Chat (RAG):** Ask questions about the repository using embeddings and vector search.
5. **User Features:** Authentication, saved history, search, and repository comparison.
6. **Production Ready:** Diagrams, code review, testing, Docker, CI/CD, and deployment.

This approach keeps the project usable after every milestone and mirrors how professional teams build software.

I can also mentor you through this project as if it's a real internship. We'll write every line ourselves (no copy-pasting), understand every concept before using it, and by the end you'll have a polished AI application that stands out on your résumé and GitHub.